

Lab 1

Part 1: Capturing HTTP Traffic

Task 1: Start Wireshark and capture packets

- Captured Wi-Fi traffic in Wireshark while browsing neverssl.com.

Task 2: Filter HTTP packets and analyze them

- Filtered with http.

Observations:

The screenshot displays the Wireshark interface with a network capture of HTTP traffic. The packet list on the left shows 24 packets, with the first 23 being GET requests and the 24th being a 304 Not Modified response. The packet details pane on the right shows the structure of the HTTP message, including the status bar at the bottom indicating 'HTTP/1.1 304 Not Modified'.

No.	Time	Source	Destination	Protocol	Length	Info
422	19.641721700	192.168.8.107	34.223.124.45	HTTP	639	GET /online/ HTTP/1.1
426	20.050878000	34.223.124.45	192.168.8.107	HTTP	180	HTTP/1.1 200 OK (text/html)
580	36.670253400	192.168.8.107	142.250.187.3	HTTP	254	GET /r/r4.cr1 HTTP/1.1
583	36.731058900	142.250.187.3	192.168.8.107	HTTP	281	HTTP/1.1 304 Not Modified
591	36.838168300	192.168.8.107	23.204.89.56	HTTP	281	GET / HTTP/1.1
593	36.935808300	23.204.89.56	192.168.8.107	HTTP	321	HTTP/1.1 304 Not Modified
601	37.056331400	192.168.8.107	86.51.94.203	HTTP	336	GET /msdownload/update/v3/static/trusted/en/authroots1.cab?feb7ae6057b37cf8 HTTP/1.1
603	37.113550300	86.51.94.203	192.168.8.107	HTTP	326	HTTP/1.1 304 Not Modified
606	37.133965100	192.168.8.107	142.250.187.3	HTTP	256	GET /r/r4.cr1 HTTP/1.1
608	37.138969600	142.250.187.3	192.168.8.107	HTTP	281	HTTP/1.1 304 Not Modified
607	37.213314600	192.168.8.107	142.250.187.3	HTTP	254	GET /r/r4.cr1 HTTP/1.1
609	37.306714700	142.250.187.3	192.168.8.107	HTTP	281	HTTP/1.1 304 Not Modified
617	37.472028000	192.168.8.107	86.51.94.211	HTTP	281	GET / HTTP/1.1
621	37.607449600	86.51.94.211	192.168.8.107	HTTP	321	HTTP/1.1 304 Not Modified
624	37.619000000	192.168.8.107	142.250.187.3	HTTP	256	GET /r/gsr4.cr1 HTTP/1.1
624	37.733555100	142.250.187.3	192.168.8.107	HTTP	280	HTTP/1.1 304 Not Modified
625	37.747168700	192.168.8.107	86.51.94.203	HTTP	342	GET /msdownload/update/v3/static/trusted/en/disallowedcerts1.cab?feb7ae6a30042295 HTTP/1.1
628	37.841801300	86.51.94.203	192.168.8.107	HTTP	326	HTTP/1.1 304 Not Modified
629	37.857780400	192.168.8.107	86.51.94.203	HTTP	336	GET /msdownload/update/v3/static/trusted/en/pinrules1.cab?3fef9df324dc93b2 HTTP/1.1
631	37.889230100	86.51.94.203	192.168.8.107	HTTP	326	HTTP/1.1 304 Not Modified
712	42.991319300	192.168.8.107	86.51.94.203	HTTP	342	GET /msdownload/update/v3/static/trusted/en/disallowedcerts1.cab?03b2e2b8a8a25713 HTTP/1.1
711	43.046771400	86.51.94.203	192.168.8.107	HTTP	326	HTTP/1.1 304 Not Modified

Figure 1 – HTTP packets filtered with "http" in Wireshark.

Method: GET. Example URL: /online/ (host 34.223.124.45) → response 200 OK (text/html). Other requests (e.g., /r/r1.crl, /r/gsr1.crl) return 304 Not Modified.

Part 2: Analyzing TCP/IP Traffic

Task 1: Filter TCP packets

- Filtered tcp.stream eq 25 and used Follow → TCP Stream.

Observations:

The screenshot shows the Wireshark network protocol analyzer interface. The top menu bar includes File, Edit, View, Go, Capture, and Analyze. The main window is titled 'Wireshark - Follow TCP Stream (tcp.stream eq 25) - شبكة Wi-Fi'. The packet list on the left shows a single packet (No. 576) from 192.168.1.1 to 192.168.1.1. The packet details pane on the right shows the following layers:

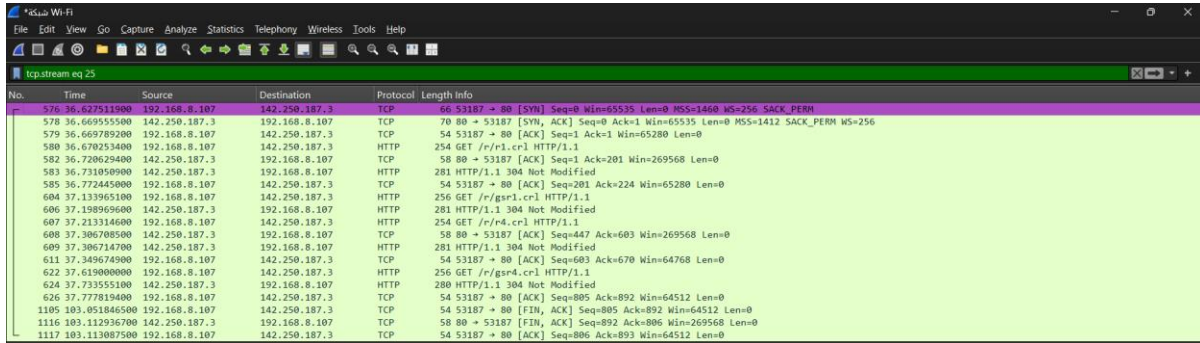
- Ethernet II, Src: Intel_Networks, Dst: Intel_Networks
- Internet Protocol Version 4, Src: 192.168.1.1, Dst: 192.168.1.1
- Transmission Control Protocol, Src Port: 4444, Dst Port: 4444

The packet bytes pane shows the raw data of the packet, which is an ICMP Echo (ping) request. The packet is 66 bytes long. The packet is captured on the 'شبكة Wi-Fi' interface.

Figure 2 – Follow TCP Stream window showing the client/server conversation with host c.pki.goog.

Task 2: Analyze TCP handshake and investigate Data Transfer and Termination

Observations:



No.	Time	Source	Destination	Protocol	Length	Info
576	36.627511900	192.168.8.107	142.250.187.3	TCP	66	53187 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
578	36.669555500	142.250.187.3	192.168.8.107	TCP	70	80 → 53187 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM WS=256
579	36.669789200	192.168.8.107	142.250.187.3	TCP	54	53187 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
580	36.670253400	192.168.8.107	142.250.187.3	HTTP	254	GET /r/r1.crl HTTP/1.1
582	36.720625400	142.250.187.3	192.168.8.107	TCP	58	80 → 53187 [ACK] Seq=1 Ack=201 Win=269568 Len=0
583	36.731050900	142.250.187.3	192.168.8.107	HTTP	281	HTTP/1.1 304 Not Modified
585	36.772445000	192.168.8.107	142.250.187.3	TCP	54	53187 → 80 [ACK] Seq=201 Ack=224 Win=65280 Len=0
604	37.133965100	192.168.8.107	142.250.187.3	HTTP	256	GET /r/gsr1.crl HTTP/1.1
606	37.150965600	142.250.187.3	192.168.8.107	HTTP	281	HTTP/1.1 304 Not Modified
607	37.213314600	192.168.8.107	142.250.187.3	HTTP	254	GET /r/r4.crl HTTP/1.1
608	37.306708500	142.250.187.3	192.168.8.107	TCP	58	80 → 53187 [ACK] Seq=447 Ack=603 Win=269568 Len=0
609	37.306714700	142.250.187.3	192.168.8.107	HTTP	281	HTTP/1.1 304 Not Modified
611	37.349674900	192.168.8.107	142.250.187.3	TCP	54	53187 → 80 [ACK] Seq=603 Ack=670 Win=64768 Len=0
622	37.619000000	192.168.8.107	142.250.187.3	HTTP	256	GET /r/gsr4.crl HTTP/1.1
624	37.733555100	142.250.187.3	192.168.8.107	HTTP	280	HTTP/1.1 304 Not Modified
626	37.777819400	192.168.8.107	142.250.187.3	TCP	54	53187 → 80 [ACK] Seq=805 Ack=892 Win=64512 Len=0
1105	103.051846500	192.168.8.107	142.250.187.3	TCP	54	53187 → 80 [FIN, ACK] Seq=805 Ack=892 Win=64512 Len=0
1116	103.112936700	142.250.187.3	192.168.8.107	TCP	58	80 → 53187 [FIN, ACK] Seq=892 Ack=806 Win=269568 Len=0
1117	103.113087500	192.168.8.107	142.250.187.3	TCP	54	53187 → 80 [ACK] Seq=806 Ack=893 Win=64512 Len=0

Figure 3 – TCP conversation (tcp.stream eq 25) showing handshake, data transfer, and termination.

Three-way handshake:

- SYN — Seq=0, Win=65535 (packet 576)
- SYN, ACK — Seq=0, Ack=1 (packet 578)
- ACK — Seq=1, Ack=1 (packet 579)

Data transfer: GET /r/r1.crl request (packet 580) → server ACK → 304 Not Modified response (packet 583), with Seq/Ack incrementing each exchange.

Termination: FIN, ACK from client Seq=805, Ack=892 (packet 1105) → FIN, ACK from server Seq=892, Ack=806 (packet 1116) → final ACK (packet 1117).

Part 3: Capturing and Analyzing UDP Traffic

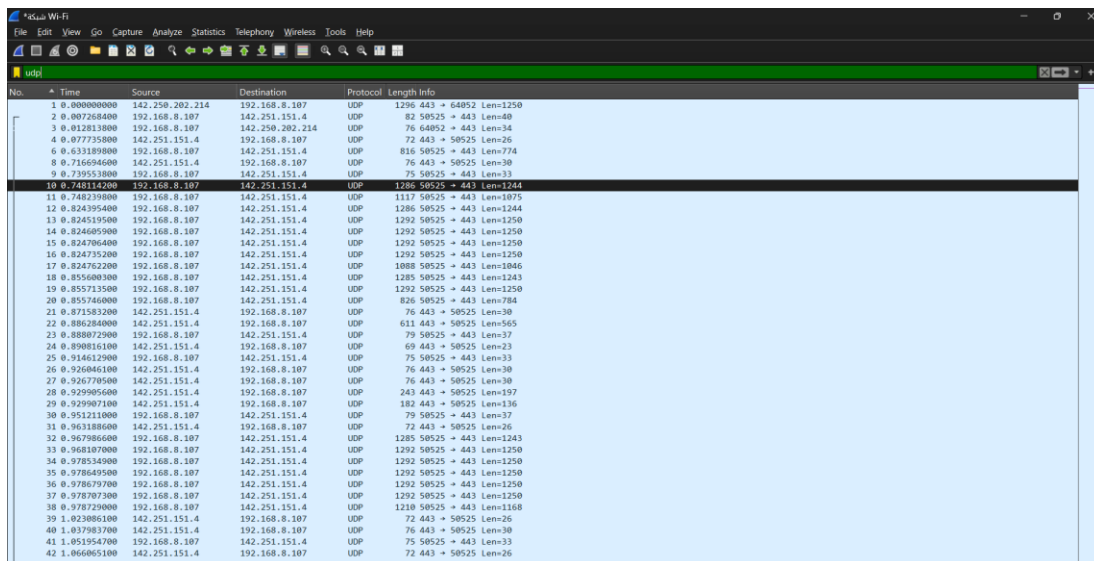
Task 1: Generate UDP traffic and capture packets

- Captured UDP traffic (QUIC/HTTPS and DNS) while browsing.

Task 2: Filter and analyze UDP packets

- Filtered with udp.

Observations:



No.	Time	Source	Destination	Protocol	Length	Info
0	0.000000000	142.250.202.214	192.168.8.107	UDP	1296	443 → 64852 Len=1250
2	0.007268400	192.168.8.107	142.251.151.4	UDP	82	50525 → 443 Len=40
3	0.012813800	192.168.8.107	142.250.202.214	UDP	76	64852 → 443 Len=34
4	0.077735800	142.251.151.4	192.168.8.107	UDP	72	443 → 50525 Len=26
6	0.633189800	192.168.8.107	142.251.151.4	UDP	816	50525 → 443 Len=774
8	0.716694600	142.251.151.4	192.168.8.107	UDP	76	443 → 50525 Len=30
9	0.729533800	192.168.8.107	142.251.151.4	UDP	75	50525 → 443 Len=33
10	0.748114200	192.168.8.107	142.251.151.4	UDP	1286	50525 → 443 Len=1244
11	0.748239800	192.168.8.107	142.251.151.4	UDP	1117	50525 → 443 Len=1075
12	0.824395400	192.168.8.107	142.251.151.4	UDP	1286	50525 → 443 Len=1244
13	0.824519500	192.168.8.107	142.251.151.4	UDP	1292	50525 → 443 Len=1250
14	0.824650800	192.168.8.107	142.251.151.4	UDP	1292	50525 → 443 Len=1250
15	0.824706400	192.168.8.107	142.251.151.4	UDP	1292	50525 → 443 Len=1250
16	0.824735200	192.168.8.107	142.251.151.4	UDP	1292	50525 → 443 Len=1250
17	0.824762200	192.168.8.107	142.251.151.4	UDP	1088	50525 → 443 Len=1046
18	0.855600300	192.168.8.107	142.251.151.4	UDP	1285	50525 → 443 Len=1243
19	0.855713500	192.168.8.107	142.251.151.4	UDP	1292	50525 → 443 Len=1250
20	0.855754000	192.168.8.107	142.251.151.4	UDP	826	50525 → 443 Len=784
21	0.871583200	142.251.151.4	192.168.8.107	UDP	76	443 → 50525 Len=30
22	0.886284000	142.251.151.4	192.168.8.107	UDP	611	443 → 50525 Len=565
23	0.888072900	192.168.8.107	142.251.151.4	UDP	79	50525 → 443 Len=37
24	0.900815100	142.251.151.4	192.168.8.107	UDP	69	443 → 50525 Len=23
25	0.914612900	192.168.8.107	142.251.151.4	UDP	75	50525 → 443 Len=33
26	0.926846100	142.251.151.4	192.168.8.107	UDP	76	443 → 50525 Len=30
27	0.926770500	142.251.151.4	192.168.8.107	UDP	76	443 → 50525 Len=30
28	0.929905600	142.251.151.4	192.168.8.107	UDP	243	443 → 50525 Len=197
29	0.929907100	142.251.151.4	192.168.8.107	UDP	182	443 → 50525 Len=136
30	0.951211800	192.168.8.107	142.251.151.4	UDP	79	50525 → 443 Len=37
31	0.963188600	142.251.151.4	192.168.8.107	UDP	72	443 → 50525 Len=26
32	0.967866600	192.168.8.107	142.251.151.4	UDP	1285	50525 → 443 Len=1243
33	0.968107800	192.168.8.107	142.251.151.4	UDP	1292	50525 → 443 Len=1250
34	0.978534000	192.168.8.107	142.251.151.4	UDP	1292	50525 → 443 Len=1250
35	0.978649500	192.168.8.107	142.251.151.4	UDP	1292	50525 → 443 Len=1250
36	0.978679700	192.168.8.107	142.251.151.4	UDP	1292	50525 → 443 Len=1250
37	0.978707300	192.168.8.107	142.251.151.4	UDP	1292	50525 → 443 Len=1250
38	0.978729800	192.168.8.107	142.251.151.4	UDP	1210	50525 → 443 Len=1168
39	1.023086100	142.251.151.4	192.168.8.107	UDP	72	443 → 50525 Len=26
40	1.037983700	142.251.151.4	192.168.8.107	UDP	76	443 → 50525 Len=30
41	1.051595400	192.168.8.107	142.251.151.4	UDP	75	50525 → 443 Len=33
42	1.066065100	142.251.151.4	192.168.8.107	UDP	72	443 → 50525 Len=26
43	1.068852200	142.251.151.4	192.168.8.107	UDP	643	443 → 50525 Len=104

Figure 4 – UDP packets filtered with "udp" (client 192.168.8.107 ↔ 142.251.151.4:443).

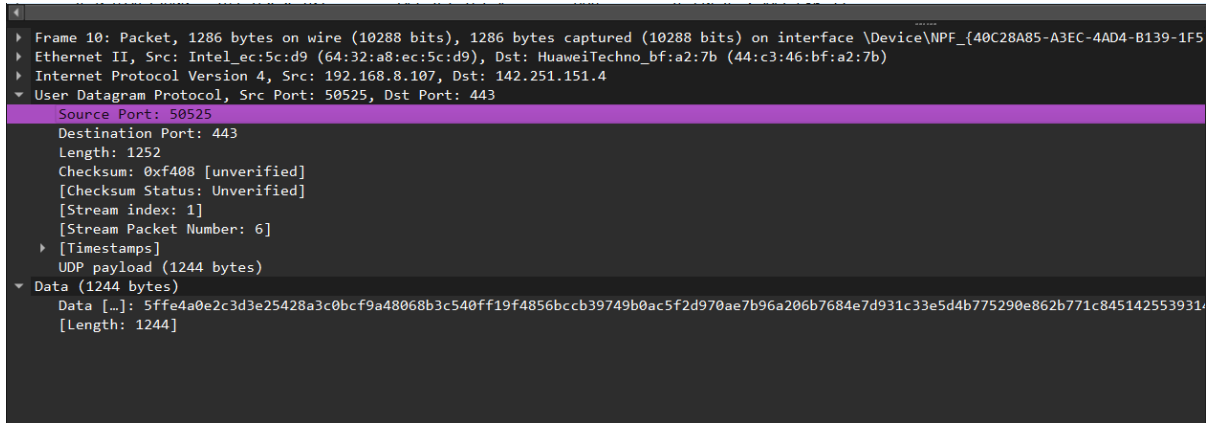


Figure 5 – UDP header detail of Frame 10 (Source Port 50525, Destination Port 443, Length 1252).

UDP header contains only Source Port, Destination Port, Length, and Checksum — no sequence/acknowledgment numbers, window size, or flags as in TCP. Packets flow immediately with no handshake, confirming UDP's simpler, lighter header.

Part 4: Comparing TCP and UDP

Task 1: Fill in the following table and provide reasons

	TCP or UDP	Reasons
Reliability and Connection Establishment	TCP	TCP performs a three-way handshake (SYN, SYN-ACK, ACK) before sending data and closes the connection with FIN/ACK packets. UDP has no handshake or teardown — packets simply start flowing.
Data Integrity and Ordering	TCP	TCP uses sequence and acknowledgment numbers on every segment to detect loss and guarantee ordered delivery. UDP's header has no such fields, so it cannot detect or correct lost or out-of-order packets.

Task 2: Identify the use cases and performance of TCP and UDP

	TCP	UDP
Use cases	Web browsing (HTTP/HTTPS), file transfer, email — traffic needing guaranteed, ordered delivery.	DNS lookups, streaming (QUIC/HTTP3), VoIP, video calls, online gaming — traffic favoring speed over guaranteed delivery.
Performance	Higher overhead: handshake, per-segment ACKs, and retransmissions add latency but guarantee correctness.	Lower overhead: no handshake/ACKs, faster and lower latency, but no delivery guarantee.